

OpenAI API Key & 첫 번째 API 호출

안전한 API 인증과 효과적인 활용 방법

API Key 관리부터 비용 계산까지 실전 가이드

LLM 활용 교육 과정

목차

01 학습 목표

- API Key 역할 이해
- 안전한 저장 방법
- 첫 API 호출 성공
- 비용 계산 능력

02 OpenAI API Key

- 인증 토큰의 정의
- 발급 절차
- 유출 위험성
- 기본 정보

03 API Key 관리

- 저장 방법 비교
- Colab Secrets 설정
- 보안 수준 차이
- 자주 발생하는 오류

04 클라이언트 & API 호출

- OpenAI 클라이언트
- chat.completions.create
- 주요 파라미터
- messages 구조

05 응답 객체 구조

- 응답 객체 접근 경로
- finish_reason 의미
- 토큰 수 확인
- 데이터 추출 방법

06 토큰과 비용 계산

- 토큰의 정의
- 단가 정보
- 비용 계산 흐름
- TokenTracker 활용

학습 목표

API Key 이해

- OpenAI API Key의 역할 설명
 - 유출 시 발생 위험 이해
- 인증 토큰의 중요성 파악
- 발급 및 관리 방법 숙지

안전한 관리

- Colab Secrets 저장 방법
- 노트북 액세스 토글 활성화
- 코드 직접 작성 위험 인식
- 보안 수준별 저장 방법 비교

API 호출 실습

- OpenAI 클라이언트 생성
- chat.completions.create 호출
 - 필수·선택 파라미터 구분
 - 응답 객체 경로 접근 성공

비용 계산

- 토큰 개념 이해
- 입력·출력 토큰 수 확인
 - 단가 기준 비용 계산
- TokenTracker 활용 방법

OpenAI API Key란?



API Key의 정의와 역할

- OpenAI 서버에 유효한 사용자임을 증명하는 인증 토큰
- Key 없이는 API 호출 불가능



발급 정보 및 형식

- 발급 위치: platform.openai.com → API Keys
- 형식: sk-로 시작하는 51자리 이상 문자열



유출 위험과 관리

- 유출 시 타인이 내 계정으로 과금 발생 가능
 - 분실 시 즉시 폐기(Revoke) 후 재발급 필요
-

API Key 발급 절차

1단계

- platform.openai.com 접속
 - 계정 로그인 완료
- 우측 상단 계정 아이콘 확인

2단계

- 계정 아이콘 클릭
- API Keys 메뉴 선택
- Key 관리 페이지 진입

3단계

- + Create new secret key 클릭
- Key 이름 입력 (예: Ilm-course)
 - 프로젝트별 구분 가능

4단계

- Create secret key 버튼 클릭
 - 생성된 Key 전체 표시
 - 딱 한 번만 확인 가능

5단계

- 표시된 Key 전체 복사
- 즉시 안전한 곳에 저장
- 창 닫기 전 반드시 보관

API Key 저장 방법 비교

코드 직접 작성

- 보안 수준: 매우 낮음 (절대 금지)
- 노트북 공유 시 Key 함께 전달
- GitHub 업로드 시 즉시 유출
- 화면 공유 중 노출 위험

변수 저장

- 보안 수준: 낮음 (비권장)
- 코드 내 변수로 저장
- 파일 공유 시 함께 노출
- 실수로 인한 유출 가능성

환경변수

- 보안 수준: 보통 (가능)
- 시스템 환경변수 활용
- 코드와 분리된 저장
- 일정 수준 보안 제공

Colab Secrets (권장)

- 보안 수준: 높음 (권장)
- 코드 밖 별도 저장소 보관
- 이름으로만 참조 가능
- 노트북 공유 시 안전

Colab Secrets 설정 방법

사이드바 접근

- 왼쪽 사이드바에서 아이콘 클릭
- 아이콘이 보이지 않으면 사이드바 펼치기
- Secrets 메뉴로 이동

보안 비밀 추가

- '+ 새 보안 비밀 추가' 버튼 클릭
- 새로운 Key 등록 화면 표시
- 이름과 값 입력 준비

Key 이름 입력

- 이름: OPENAI_API_KEY 입력
- 대소문자 정확히 입력 필수
- 코드에서 참조할 이름

Key 값 입력

- 발급받은 Key 전체 붙여넣기
- sk-로 시작하는 전체 문자열
- 공백 없이 정확히 입력

액세스 활성화

- 노트북 액세스 토글 활성화
- 파란색으로 변경 확인 필수
- 미활성화 시 코드에서 읽기 불가

openai.OpenAI() 클라이언트



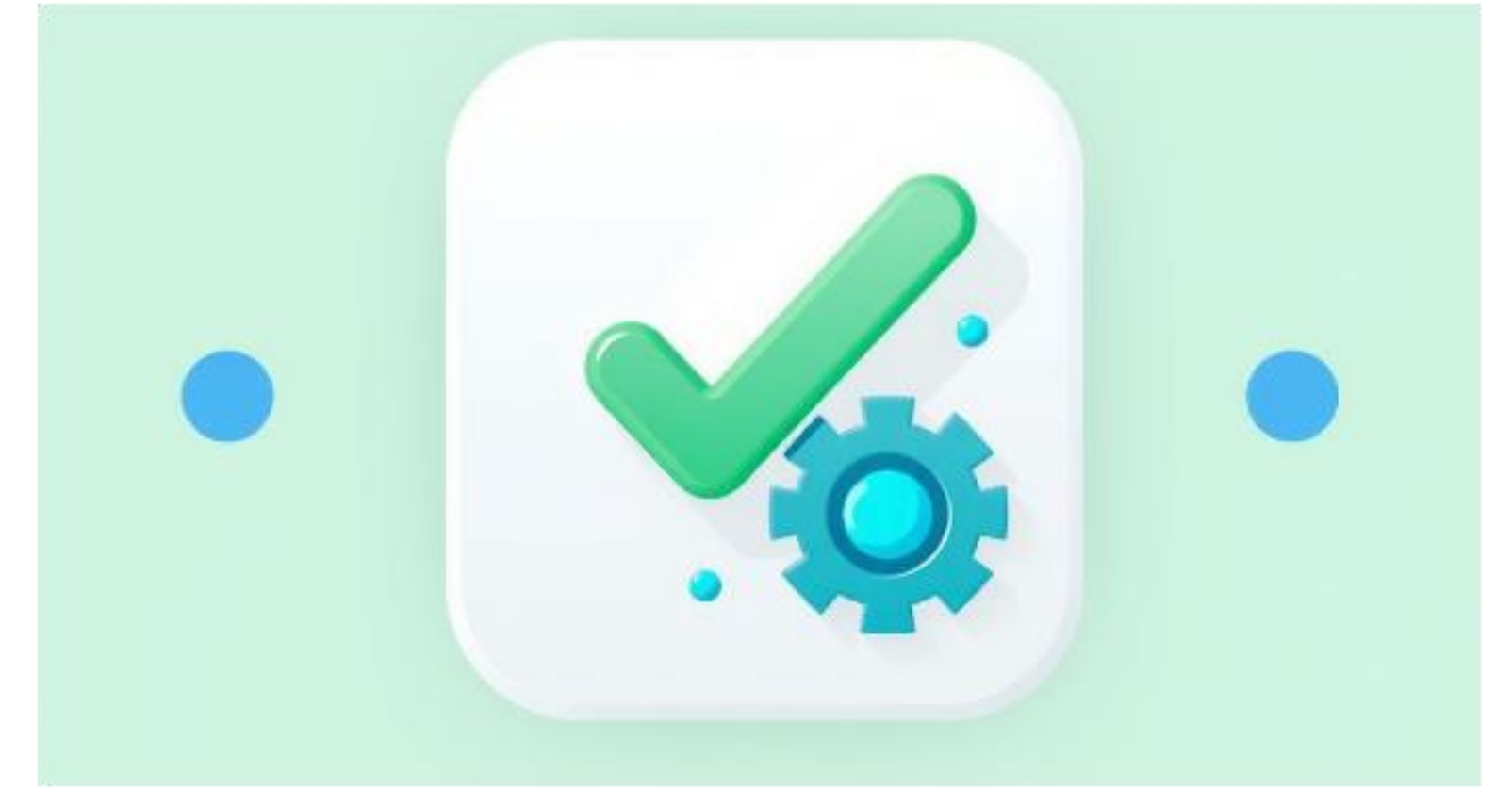
클라이언트의 역할

- OpenAI API와 통신하기 위한 연결 객체
- API Key 인증 정보를 한 번만 설정
- 이후 모든 API 호출은 이 객체를 통해 실행
- 전역 변수로 생성하여 재사용
- 매번 생성 시 인증 과정 반복으로 비효율



자주 발생하는 오류

- AuthenticationError: Key가 잘못되거나 없음
- SecretNotFoundError: Secrets 이름 불일치
- NotFoundError: 노트북 액세스 토글 미활성화
- RateLimitError: 무료 플랜 한도 초과
- 대부분 Secrets 설정 문제로 발생



오류 해결 방법

- Secrets 이름이 코드와 정확히 일치하는지 확인
- 대소문자 구분 철저히 체크
- 노트북 액세스 토글 파란색 확인
- Key 값이 sk-로 시작하는지 확인
- 문제 지속 시 Key 재발급 후 재설정

chat.completions.create 주요 파라미터

필수 파라미터

- model: 사용할 모델 이름 (예: gpt-4o-mini)
- messages: 대화 내용을 담은 리스트
- 이 두 파라미터는 반드시 지정 필요
- 누락 시 API 호출 실패

자주 조절하는 파라미터

- temperature: 응답 다양성 조절 (0.0~2.0)
- max_tokens: 최대 출력 토큰 수 제한
- 작업 특성에 따라 값 조정 필요
- 기본값 사용 시 생략 가능

선택 파라미터

- top_p: 누클리어스 샘플링 (0.0~1.0)
- n: 응답 후보 개수 (과금 주의)
- stream: 스트리밍 출력 여부
- 필요에 따라 선택적으로 사용

messages 구조

- role: system, user, assistant 중 선택
- content: 실제 메시지 내용
- system은 AI 역할 설정, user는 질문
- assistant는 이전 응답 기록

응답(Response) 객체 구조

실제 응답 텍스트 추출

- response.choices[0].message.content
 - AI가 생성한 실제 응답 텍스트
 - 가장 자주 사용하는 접근 경로
 - 문자열(str) 형태로 반환
 - 출력 또는 후처리에 활용

토큰 사용량 확인

- response.usage.prompt_tokens
- response.usage.completion_tokens
- response.usage.total_tokens
 - 입력·출력·전체 토큰 수
 - 비용 계산 필수 정보

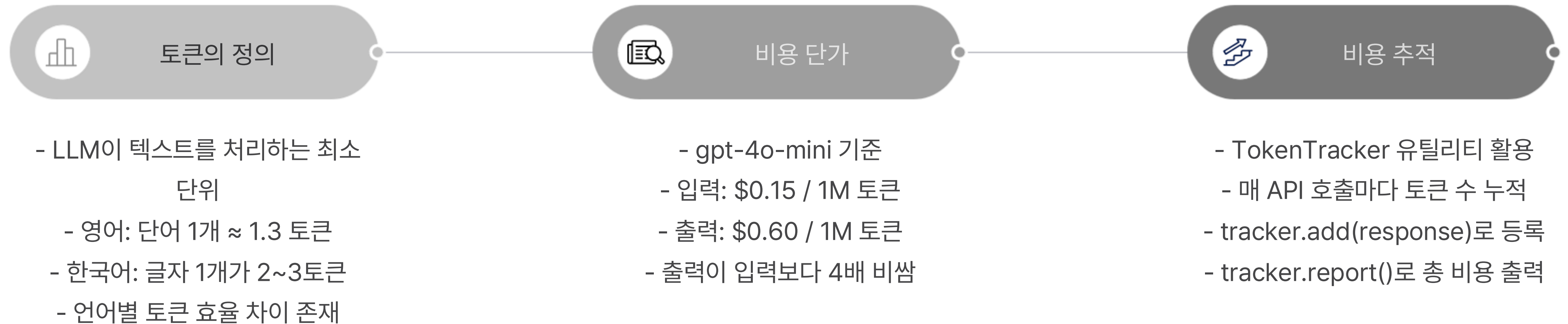
종료 이유 확인

- response.choices[0].finish_reason
 - 응답 종료 방식 표시
 - stop·length·content_filter 값
 - 응답 완성도 판단 기준
 - 오류 대응에 활용

finish_reason 값의 의미

- stop: 정상 종료(모델 완료)
- length: max_tokens 초과로 중단
- content_filter: 안전 정책 차단
- length 발생 시 토큰 수 증가 필요
- JSON 응답 시 주의

토큰(Token)과 비용 계산



토큰은 LLM의 기본 단위이며 한국어는 토큰 효율이 낮아 비용이 더 듭니다. gpt-4o-mini는 호출당 저렴하지만 대량 호출 시 비용이 누적됩니다. `response.usage`로 입력·출력 토큰을 확인해 단가를 곱해 총비용을 계산합니다. TokenTracker는 여러 호출의 토큰을 누적해 총비용을 추적하는 도구로 파이프라인에서 활용됩니다.

실습 체크포인트 및 정리

이번 실습에서는 OpenAI API Key를 안전하게 발급하고 Colab Secrets에 저장하는 방법을 학습했습니다. `openai.OpenAI()` 클라이언트를 생성하여 첫 API 호출에 성공하고, 응답 객체에서 텍스트와 토큰 정보를 정확히 추출했습니다. 또한 비용 계산 방법과 `TokenTracker`를 통한 누적 비용 관리 방법을 익혔습니다.
